Image Transformations

Abstract

Image transformations describe how points, lines, and pixel values move between images. We develop the hierarchy from translations to projective transformations, identify the geometry each family preserves, and derive how homographies act on points and lines. Planar scenes and cameras rotating about a fixed centre provide two important sources of homographies. We then recover pose from a calibrated view of a plane. We also explain how inverse warping and interpolation turn a geometric mapping into a new image.

Contents

1 From Image Formation to Image Transformations **2**

2 The Hierarchy of Image Transformations **3**

3 Projective Geometry: Cross Ratio and Lines **7**

4 Homographies from Cameras and Planes **9**

5 From Geometric Mappings to Warped Images **12**

6 Summary of Core Formulas **15**

7 Common Pitfalls **15**

1 From Image Formation to Image Transformations

A camera maps three-dimensional scene points to a two-dimensional image. An image transformation instead maps positions in one image to positions in another. Such mappings let us align photographs, insert graphics onto a surface, or compensate for camera rotation.

Two questions must be kept separate: *where does a point move?* and *what value should an output pixel receive?* The first is geometric. The second requires sampling the input image, usually between its pixel centres.

1.1 Points and homogeneous coordinates

We use column vectors and apply matrices on the left. Write an image point as the column vector, $\mathbf{x} = (x, y)^\top \in \mathbb{R}^2$. Here $\mathbb{R}^2$ is the set of real two-component vectors, and $^\top$ denotes transpose. Its homogeneous representative is

$$\widetilde{\mathbf{x}} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}.$$

A tilde distinguishes a three-component homogeneous vector from a two-component image point. Nonzero scalar multiples describe the same point:

$$\boxed{\widetilde{\mathbf{x}} \sim \lambda\widetilde{\mathbf{x}}, \qquad \lambda \neq 0.}$$

The symbol, $\sim$, means equality up to a nonzero scale. In particular, $(2, 4, 2)^\top$ and $(1, 2, 1)^\top$ represent the same image position.

For a general homogeneous vector, $\mathbf{q} = (q_1, q_2, q_3)^\top$, the dehomogenization function, $\pi(\cdot)$, is

$$\boxed{\pi(\mathbf{q}) = \begin{pmatrix} q_1/q_3 \\ q_2/q_3 \end{pmatrix}, \qquad q_3 \neq 0.}$$

The third component is a scale coordinate, not an extra image coordinate. To understand a zero last component, let $\mathbf{d} = (d_x, d_y)^\top \neq \mathbf{0}$ and move along $\mathbf{x}(t) = t\mathbf{d}$. For $t > 0$, rescale its homogeneous representative:

$$\begin{pmatrix} td_x \\ td_y \\ 1 \end{pmatrix} \sim \begin{pmatrix} d_x \\ d_y \\ 1/t \end{pmatrix} \xrightarrow[t\to\infty]{} \begin{pmatrix} d_x \\ d_y \\ 0 \end{pmatrix}.$$

The image position recedes without bound, but its rescaled representative has a finite limit. This motivates calling $(d_x, d_y, 0)^\top$ a *point at infinity*: it records a line direction rather than a finite pixel position. Opposite directions give equivalent representatives. The all-zero vector does not represent a point.

2 The Hierarchy of Image Transformations

2.1 Homogeneous matrices and translation

A 2×2 matrix maps the origin to itself, so it cannot represent a nonzero translation. Homogeneous coordinates put translation and linear transformations into one matrix expression:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11}x + a_{12}y + t_x \\ a_{21}x + a_{22}y + t_y \\ 1 \end{pmatrix}.$$

The upper-left 2×2 block applies the linear transformation, and the last column adds the translation. The last row keeps the homogeneous coordinate equal to one.

A translation adds the same displacement, $\mathbf{t} = (t_x, t_y)^\top$, to every point:

$$\boxed{\mathbf{x}' = \mathbf{x} + \mathbf{t}, \qquad \widetilde{\mathbf{x}}' = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \widetilde{\mathbf{x}}.}$$

There are two degrees of freedom, one per translation component. Differences between points are unchanged, so lengths, angles, areas, and line directions are preserved.

2.2 Euclidean transformations and similarities

A Euclidean transformation combines a rotation with a translation. Here x increases to the right and y upward, so a positive angle gives a counterclockwise rotation:

$$\mathbf{R}(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}, \qquad \mathbf{R}^\top\mathbf{R} = \mathbf{I}, \qquad \det\mathbf{R} = 1.$$

$$\boxed{\mathbf{x}' = \mathbf{R}\mathbf{x} + \mathbf{t}, \qquad \widetilde{\mathbf{x}}' = \begin{pmatrix} \cos\theta & -\sin\theta & t_x \\ \sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{pmatrix} \widetilde{\mathbf{x}}.}$$

Here $\mathbf{I}$ is the identity matrix and det denotes determinant. The angle and two translation components give three degrees of freedom. Write $\|\mathbf{d}\| = \sqrt{d_x^2 + d_y^2}$ for the Euclidean length of a two-component vector. To see why distances are preserved, let $\mathbf{d} = \mathbf{x}_a - \mathbf{x}_b$. Translation cancels in the difference, leaving

$$\left\|\mathbf{x}'_a - \mathbf{x}'_b\right\|^2 = (\mathbf{R}\mathbf{d})^\top(\mathbf{R}\mathbf{d}) = \mathbf{d}^\top\mathbf{R}^\top\mathbf{R}\mathbf{d} = \|\mathbf{d}\|^2 .$$

The same dot-product identity preserves angles. Areas are preserved as well. Here, Euclidean transformations mean rotations and translations; reflections are excluded. Rotation can change a line's direction without reflecting the shape: three labelled vertices traced counterclockwise around a triangle remain counterclockwise after rotation and translation.

A similarity adds a uniform scale, $\alpha > 0$:

$$\boxed{\mathbf{x}' = \alpha\mathbf{R}\mathbf{x} + \mathbf{t}, \qquad \widetilde{\mathbf{x}}' = \begin{pmatrix} \alpha\cos\theta & -\alpha\sin\theta & t_x \\ \alpha\sin\theta & \alpha\cos\theta & t_y \\ 0 & 0 & 1 \end{pmatrix} \widetilde{\mathbf{x}}.}$$

Its four degrees of freedom are scale, angle, and two translations. Lengths are multiplied by α and areas by α^2. Angles, ratios of lengths, and ratios of areas are preserved. Absolute lengths and areas generally are not.

Example: order matters. Rotate $\mathbf{x} = (1, 0)^\top$ by 90° and then translate by $(2, 0)^\top$. The result is $(2, 1)^\top$. Translating first gives $(3, 0)^\top$, whose rotation is $(0, 3)^\top$. Matrix multiplication records this order: the rightmost operation acts first.

2.3 Affine transformations

An affine transformation allows an arbitrary nonsingular 2×2 matrix, $\mathbf{A}$.[1] Its homogeneous form is

$$\boxed{\mathbf{x}' = \mathbf{A}\mathbf{x} + \mathbf{t}, \qquad \widetilde{\mathbf{x}}' = \begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \widetilde{\mathbf{x}}, \quad \mathbf{A} = \begin{pmatrix} a & b \\ d & e \end{pmatrix}, \quad \mathbf{t} = \begin{pmatrix} c \\ f \end{pmatrix}.}$$

[1] A square matrix, $\mathbf{M}$, is *nonsingular* if and only if $\det\mathbf{M} \neq 0$, equivalently if its inverse, $\mathbf{M}^{-1}$, exists. If $\det\mathbf{M} = 0$, it is *singular*: some nonzero vector maps to zero, so distinct inputs can have the same output and the mapping cannot be uniquely reversed.

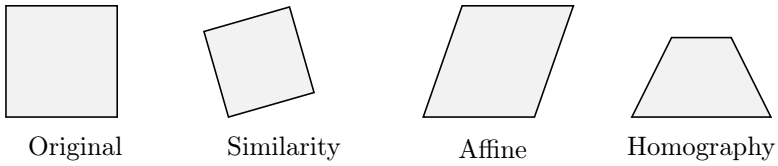


Figure 1: A square under representative transformations. Similarity retains its angles and proportions; affine shear retains parallel sides; a general homography need not retain parallelism. The panels use a common drawing scale.

The six free entries give six degrees of freedom. Affine transformations include shear and unequal scaling along different directions, as well as similarities.

Parameterize a line by $\mathbf{x}(s) = \mathbf{p} + s\mathbf{d}$, where $\mathbf{d} \neq \mathbf{0}$. Its transformed points satisfy

$$\mathbf{x}'(s) = \mathbf{A}\mathbf{p} + \mathbf{t} + s\mathbf{A}\mathbf{d}.$$

This is still a line. Parallel lines share a direction, so their transformed directions are also shared. The parameter, s, is unchanged, which preserves ratios of directed distances along a line, including midpoints. General angles and lengths are not preserved.

Figure 1 shows the increasing freedom to deform a square. More parameters allow more image configurations, but they also require more information to estimate reliably.

2.4 Projective transformations: homographies

A homography is represented by a nonsingular 3×3 matrix, $\mathbf{H}$:

$$\boxed{w \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}}_{\mathbf{H}} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}.}$$

Reading the three rows gives $wx' = ax + by + c$, $wy' = dx + ey + f$, and $w = gx + hy + i$. Thus

$$\boxed{x' = \frac{ax + by + c}{gx + hy + i}, \qquad y' = \frac{dx + ey + f}{gx + hy + i}.}$$

For an affine transformation, the homogeneous denominator is constant and can be scaled to one. A general projective transformation lets it depend on position. This position-dependent division produces perspective effects: parallel lines can converge toward a vanishing point. These formulas apply wherever the denominator is nonzero. A zero denominator sends a finite point to infinity; it does not by itself make the matrix singular.

There are nine matrix entries but only eight degrees of freedom: multiplying every entry by the same nonzero number leaves the mapping unchanged. One may set $h_{33} = 1$ only when that entry is nonzero. The choice of matrix scale is not an additional geometric parameter.

A homography preserves lines and cross ratio, but generally not angles, lengths, parallelism, or ratios of areas.

Table 1: Transformation families, degrees of freedom, and preserved geometry.

Family	DoF	Preserved geometry
Translation	2	Lines, line directions, lengths, angles, and areas
Euclidean	3	Lines, lengths, angles, and areas
Similarity	4	Lines, angles, length ratios, and area ratios
Affine	6	Lines, parallelism, ratios along a line, and area ratios
Homography	8	Lines, incidence, and cross ratio

Example: a finite point mapped to infinity. For

$$\mathbf{H} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & 0 & 1 \end{pmatrix}, \qquad \mathbf{H} \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix},$$

the point $(2, 1)^\top$ maps to $(1, 1/2)^\top$. Every point $\mathbf{x} = (-2, y)^\top$, with $y \in \mathbb{R}$, has representative $\widetilde{\mathbf{x}} = (-2, y, 1)^\top$ and maps to $\mathbf{H}\widetilde{\mathbf{x}} = (-2, y, 0)^\top$, a point at infinity.

2.5 Nested families and composition

Moving from left to right adds degrees of freedom and allows more general transformations. Each family contains all transformations in the preceding families; $\subset$ denotes this inclusion:

$$\text{translation} \subset \text{Euclidean} \subset \text{similarity} \subset \text{affine} \subset \text{homography}.$$

Table 1 compares the invertible models used here. The degrees of freedom (DoF) count continuous parameters.

Beyond choosing a transformation family, we may need to combine several transformations, such as a rotation followed by a translation. Homogeneous matrices let us represent the entire sequence with one matrix. Let $\mathbf{H}_1$ map the original coordinates to intermediate coordinates, and let $\mathbf{H}_2$ map those to the final coordinates. Then

$$\widetilde{\mathbf{x}}' \sim \mathbf{H}_1\widetilde{\mathbf{x}}, \qquad \widetilde{\mathbf{x}}'' \sim \mathbf{H}_2\widetilde{\mathbf{x}}' \quad \Longrightarrow \quad \widetilde{\mathbf{x}}'' \sim \underbrace{\mathbf{H}_2\mathbf{H}_1}_{\mathbf{H}}\widetilde{\mathbf{x}}.$$

The product, $\mathbf{H}$, is a 3×3 matrix with inverse

$$\mathbf{H}^{-1} = \mathbf{H}_1^{-1}\mathbf{H}_2^{-1}.$$

Thus the composition is itself a homography; the rightmost matrix acts first. Intermediate homogeneous scales cancel upon dehomogenization, so multiply the matrices and divide by the final third component once, provided it is nonzero. Resample the image only once to avoid blur from repeated interpolation.

3 Projective Geometry: Cross Ratio and Lines

3.1 Cross ratio of four collinear points

Perspective changes distances along a line but preserves their cross ratio. Let $\mathbf{p}_1, \ldots, \mathbf{p}_4$ be four distinct collinear points, and let $s_1, \ldots, s_4$ be their scalar coordinates along that line. Define the signed cross ratio by

$$\boxed{C = \frac{(s_3 - s_1)(s_4 - s_2)}{(s_3 - s_2)(s_4 - s_1)}.}$$

If the points occur in the order $1, 2, 3, 4$ along the line, this is also

$$C = \frac{\|\mathbf{p}_3 - \mathbf{p}_1\| \, \|\mathbf{p}_4 - \mathbf{p}_2\|}{\|\mathbf{p}_3 - \mathbf{p}_2\| \, \|\mathbf{p}_4 - \mathbf{p}_1\|}.$$

For other orderings, the norm expression gives $|C|$; signed coordinates retain the sign.

Point ordering matters. Use matching labels in both views. Permuting labels can change the value: $s_1, s_2, s_3, s_4 = 0, 1, 2, 3$ gives $C = 4/3$; swapping labels 1 and 2 gives $3/4$.

3.2 Representing an image line

The line equation, $ax + by + c = 0$, can be written as an incidence relation between a point and a line:

$$\boxed{\mathbf{l} = \begin{pmatrix} a \\ b \\ c \end{pmatrix}, \qquad \mathbf{l}^\top \widetilde{\mathbf{x}} = 0.}$$

Here $\mathbf{l}$ is a nonzero homogeneous line vector, defined up to nonzero scale. Its first two entries form the normal to the line in the image. In three-dimensional homogeneous-coordinate space, the full vector is normal to the plane of representatives of points on that line.

The symbol, $\times$, denotes the three-dimensional cross product. Two distinct homogeneous points define a line through their cross product; two distinct lines define their intersection in the same way:

$$\mathbf{l} \sim \widetilde{\mathbf{x}}_1 \times \widetilde{\mathbf{x}}_2, \qquad \widetilde{\mathbf{x}} \sim \mathbf{l}_1 \times \mathbf{l}_2.$$

The cross product is orthogonal to both inputs, so each construction satisfies the required incidence equations. Parallel image lines intersect at a point with last coordinate zero.

3.3 Why lines transform by the inverse transpose

Points transform with $\mathbf{H}$. We seek a transformation of the line, $\mathbf{l}$, that keeps every transformed point on the transformed line. If $\widetilde{\mathbf{x}}' \sim \mathbf{H}\widetilde{\mathbf{x}}$, substitute $\widetilde{\mathbf{x}} \sim \mathbf{H}^{-1}\widetilde{\mathbf{x}}'$ into its equation:

$$\begin{aligned} 0 &= \mathbf{l}^\top \widetilde{\mathbf{x}} \\ &= \mathbf{l}^\top \mathbf{H}^{-1} \widetilde{\mathbf{x}}' \\ &= (\mathbf{H}^{-\top} \mathbf{l})^\top \widetilde{\mathbf{x}}'. \end{aligned}$$

Homogeneous scales do not affect an equation whose right side is zero. Consequently,

$$\boxed{\mathbf{l}' \sim \mathbf{H}^{-\top}\mathbf{l}, \qquad \mathbf{H}^{-\top} = (\mathbf{H}^{-1})^{\top}.}$$

Equivalently, inserting $\mathbf{H}^{\top}\mathbf{H}^{-\top} = \mathbf{I}$ into $\widetilde{\mathbf{x}}^{\top}\mathbf{l} = 0$ gives $(\mathbf{H}\widetilde{\mathbf{x}})^{\top}\mathbf{H}^{-\top}\mathbf{l} = 0$. Points and lines transform differently so that incidence is preserved.

Example: translating a vertical line. The line $x = 2$ has vector $\mathbf{l} = (1, 0, -2)^{\top}$. Translate every point by $(3, -1)^{\top}$:

$$\mathbf{H} = \begin{pmatrix} 1 & 0 & 3 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \mathbf{H}^{-\top}\mathbf{l} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -3 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ -5 \end{pmatrix}.$$

The new line is $x' = 5$, as expected. Multiplying the line by $\mathbf{H}$ instead would not produce this result.

4 Homographies from Cameras and Planes

4.1 Case 1: From a world plane to its image

Let a world point be $\mathbf{p} = (X, Y, Z)^\top$, with homogeneous representative $\widetilde{\mathbf{p}} = (X, Y, Z, 1)^\top$. The camera coordinates are

$$\mathbf{p}_c = \mathbf{R}\mathbf{p} + \mathbf{t}, \qquad \mathbf{R} = [\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{r}_3] \in \mathbb{R}^{3\times 3}.$$

The rotation matrix, $\mathbf{R}$, satisfies $\mathbf{R}^\top\mathbf{R} = \mathbf{I}$ and $\det \mathbf{R} = 1$. The translation vector, $\mathbf{t} \in \mathbb{R}^3$, is the world origin expressed in camera coordinates; it is not the camera centre in world coordinates.

The intrinsic matrix, $\mathbf{M}_{\text{int}} \in \mathbb{R}^{3\times 3}$, converts a camera ray to homogeneous image coordinates. We write the full projection as

$$\boxed{w\widetilde{\mathbf{x}} = \mathbf{M}_{\text{int}} \underbrace{[\mathbf{R} \mid \mathbf{t}]}_{\mathbf{M}_{\text{ext}}} \widetilde{\mathbf{p}}.}$$

The 3×4 extrinsic matrix, $\mathbf{M}_{\text{ext}}$, changes coordinates before projection. Lens distortion is assumed absent or already corrected.

The camera looks along negative Z_c, and visible points have $Z_c < 0$. The signed image-plane coordinate is $f < 0$. Another common computer-vision convention uses $Z_c > 0$ and $f > 0$; the geometry is equivalent when the camera frame, image axes, and projection equations use the chosen convention consistently. Before pixel-coordinate conversion, one may use $\text{diag}(f, f, 1)$ for the intrinsic matrix, where diag constructs a diagonal matrix from the listed entries. Pixel scale, principal-point offset, and any image-axis reversal are incorporated into $\mathbf{M}_{\text{int}}$.

Step 1: choose coordinates on the plane. Place the world plane at $Z = 0$ and represent a point on it by $\widetilde{\mathbf{u}} = (X, Y, 1)^\top$. This is a choice of world frame, not an assumption that the plane faces the camera.

Step 2: remove the zero contribution.

$$\begin{aligned} w\widetilde{\mathbf{x}} &= \mathbf{M}_{\text{int}}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{r}_3\ \mathbf{t}] \begin{pmatrix} X \\ Y \\ 0 \\ 1 \end{pmatrix} \\ &= \mathbf{M}_{\text{int}}(\mathbf{r}_1 X + \mathbf{r}_2 Y + \mathbf{t}) \\ &= \mathbf{M}_{\text{int}}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]\widetilde{\mathbf{u}}. \end{aligned}$$

Hence a 3D-to-2D projection restricted to a plane becomes a 2D-to-2D mapping:

$$\boxed{\widetilde{\mathbf{x}} \sim \mathbf{H}\widetilde{\mathbf{u}}, \qquad \mathbf{H} = \mathbf{M}_{\text{int}}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}].}$$

This is a homography when the camera centre is not on the plane. Indeed, $\det[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}] = (\mathbf{r}_1 \times \mathbf{r}_2)^\top \mathbf{t} = \mathbf{r}_3^\top \mathbf{t}$. If this vanishes, the plane passes through the camera centre and its image collapses to a line.

4.2 Recovering pose from a calibrated planar view

A known homography from a planar target encodes its orientation and position relative to the camera. Assume that the intrinsic matrix is known and the target coordinates, (X, Y), use a known metric scale.

Step 1: remove the intrinsics. A homography, $\widehat{\mathbf{H}}$, has an arbitrary nonzero scale, s:

$$\widehat{\mathbf{H}} = s\mathbf{M}_{\text{int}}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}].$$

Define

$$\mathbf{B} = \mathbf{M}_{\text{int}}^{-1}\widehat{\mathbf{H}} = [\mathbf{b}_1\ \mathbf{b}_2\ \mathbf{b}_3] = s[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}].$$

This cancels the image-coordinate calibration and leaves scaled camera-frame quantities.

Step 2: recover one common scale. Rotation columns have unit norm and are orthogonal. In exact data,

$$\|\mathbf{b}_1\| = \|\mathbf{b}_2\| = |s|, \qquad \mathbf{b}_1^\top \mathbf{b}_2 = 0.$$

Choose $\lambda = \pm 1/\|\mathbf{b}_1\|$ and set

$$\boxed{\mathbf{r}_1 = \lambda\mathbf{b}_1, \qquad \mathbf{r}_2 = \lambda\mathbf{b}_2, \qquad \mathbf{t} = \lambda\mathbf{b}_3.}$$

The same multiplier must be applied to all three columns. Normalizing each column independently would destroy the translation magnitude. Select the sign for which the observed target points satisfy the chosen visibility condition, here $Z_c < 0$.

Step 3: recover the missing rotation column. The right-handed orthonormal frame determines the third column:

$$\boxed{\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \qquad \mathbf{R} = [\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{r}_3].}$$

The camera centre in world coordinates follows by setting $\mathbf{p}_c = \mathbf{0}$:

$$\mathbf{0} = \mathbf{R}\mathbf{c} + \mathbf{t} \quad \Longrightarrow \quad \mathbf{c} = -\mathbf{R}^\top\mathbf{t}.$$

Translation is expressed in the target's length units. Without a known target size, its absolute metric magnitude is not determined.

Measured data. Noisy correspondences generally yield columns that are not exactly orthogonal. Use a common scale based on both column norms, then enforce a proper rotation rather than treating the raw columns as exact. Finally, check that the reconstructed pose projects the target close to the measured image points. Nearly edge-on views make this recovery sensitive to measurement error.

4.3 Case 2: Two images of the same plane

Let $\mathbf{H}_1$ and $\mathbf{H}_2$ map the same plane coordinates to images 1 and 2:

$$\widetilde{\mathbf{x}}_1 \sim \mathbf{H}_1\widetilde{\mathbf{u}}, \qquad \widetilde{\mathbf{x}}_2 \sim \mathbf{H}_2\widetilde{\mathbf{u}}.$$

Invert the second relation and substitute it into the first:

$$\widetilde{\mathbf{u}} \sim \mathbf{H}_2^{-1}\widetilde{\mathbf{x}}_2 \quad \Longrightarrow \quad \widetilde{\mathbf{x}}_1 \sim \mathbf{H}_1\mathbf{H}_2^{-1}\widetilde{\mathbf{x}}_2.$$

Thus the image-to-image homography is

$$\boxed{\mathbf{H}_{1\leftarrow 2} = \mathbf{H}_1\mathbf{H}_2^{-1}.}$$

The subscript states the mapping direction: image 2 to image 1. Reversing the direction requires the inverse matrix, $\mathbf{H}_{2\leftarrow 1} = \mathbf{H}_2\mathbf{H}_1^{-1}$.

This explains why a graphic can be attached to a planar surface. A homography between two views of a playing field aligns points on the field. A player's head lies above that plane, so the same mapping can stretch or displace the player. The plane model does not describe arbitrary three-dimensional structure.

4.4 Case 3: Pure camera rotation

There is another important case in which scene points share one homography, even when they are not coplanar: two cameras with the same centre of projection. Choose that centre as the world origin and image 2's frame as the world orientation. Let $\mathbf{R}$ rotate coordinates from frame 2 to frame 1. With identical intrinsics,

$$w_1\widetilde{\mathbf{x}}_1 = \mathbf{M}_{\text{int}}\mathbf{R}\mathbf{p}, \qquad w_2\widetilde{\mathbf{x}}_2 = \mathbf{M}_{\text{int}}\mathbf{p}.$$

Solve the second equation for the scene point and substitute:

$$\begin{aligned}\mathbf{p} &= w_2\mathbf{M}_{\text{int}}^{-1}\widetilde{\mathbf{x}}_2,\\ w_1\widetilde{\mathbf{x}}_1 &= w_2\mathbf{M}_{\text{int}}\mathbf{R}\mathbf{M}_{\text{int}}^{-1}\widetilde{\mathbf{x}}_2.\end{aligned}$$

The depth-dependent scales disappear in homogeneous coordinates:

$$\boxed{\widetilde{\mathbf{x}}_1 \sim \mathbf{H}_{1\leftarrow 2}\widetilde{\mathbf{x}}_2, \qquad \mathbf{H}_{1\leftarrow 2} = \mathbf{M}_{\text{int}}\mathbf{R}\mathbf{M}_{\text{int}}^{-1}.}$$

With different intrinsics, replace the last expression by $\mathbf{M}_{\text{int},1}\mathbf{R}\mathbf{M}_{\text{int},2}^{-1}$. This maps rays between two image coordinate systems without knowing scene depth.

Rotation must be about the centre of projection. Camera translation generally introduces depth-dependent parallax, so a single homography no longer aligns a nonplanar scene exactly. This distinction matters in panorama construction and video stabilization.

5 From Geometric Mappings to Warped Images

5.1 Forward warping and its holes

Let $I(x, y)$ denote an input image, with integer pixel centres. A geometric mapping, $T(\cdot)$, sends source positions to output positions. For a transformation represented by a 3×3 matrix, $\mathbf{T}$,

$$\mathbf{x}' = T(\mathbf{x}) = \pi(\mathbf{T}\widetilde{\mathbf{x}}).$$

Here π converts homogeneous coordinates to image coordinates: $\pi(\mathbf{q}) = (q_1/q_3, q_2/q_3)^\top$, provided $q_3 \neq 0$.

A simple forward warp visits each input pixel, computes its destination, rounds that position to an output pixel, and copies the value there. This does not guarantee that every output pixel is assigned a value. Some destinations receive several samples; others receive none.

For example, the one-dimensional scaling $x' = 2x$ sends integer source positions to even integer output positions. A direct copy leaves all odd positions empty. These holes are caused by discrete sampling, even though the continuous transformation is invertible.

5.2 Inverse warping

Instead, visit every output pixel and ask which source position supplies its value. For an invertible transformation,

$$\boxed{\mathbf{x} = T^{-1}(\mathbf{x}'), \qquad I_{\text{out}}(\mathbf{x}') = I_{\text{in}}\big(T^{-1}(\mathbf{x}')\big).}$$

When the mapping has an invertible matrix representation, compute

$$\mathbf{q} = \mathbf{T}^{-1} \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix}, \qquad (x, y) = \left(\frac{q_1}{q_3}, \frac{q_2}{q_3} \right).$$

The image on the right of the boxed expression must be evaluated at a generally noninteger position. An interpolation rule supplies that value.

Inverse warping avoids holes caused by skipping output pixels. It does not create data outside the source image, reveal occluded surfaces, or recover detail lost during acquisition. If $q_3 = 0$, the source position is at infinity. In a numerical implementation, positions with a near-zero denominator or outside the valid source domain need an explicit invalid-value or background policy.

A minimal inverse-warp procedure.

1. Choose the output image size and its coordinate origin.
2. For each output pixel centre, apply $\mathbf{T}^{-1}$ and dehomogenize.
3. If the source position is valid, interpolate the input image there.
4. Otherwise, assign a background value and mark the pixel invalid if a mask is needed.

The geometric convention here is source-to-output for $\mathbf{T}$. A software routine may accept either direction, so its matrix convention must be checked before supplying an inverse.

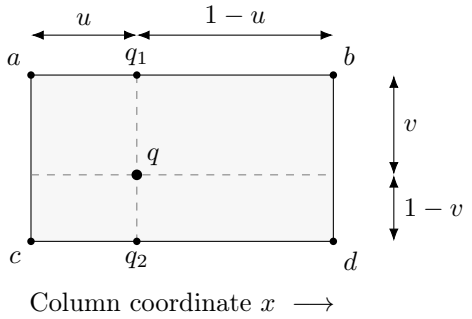


Figure 2: Bilinear interpolation within a unit pixel cell. Interpolate horizontally on each row to obtain q_1 and q_2, then vertically to obtain q at the query position. Row coordinates increase downward.

5.3 Nearest-neighbour and bilinear interpolation

Use x for the column coordinate, increasing to the right, and y for the row coordinate, increasing downward. With these image axes, the rotation matrix in Section 2.2 appears clockwise for a positive angle; its algebra is unchanged. At a fractional location, (x, y), nearest-neighbour interpolation uses the closest pixel centre:

$$\widehat{I}_{\mathrm{NN}}(x, y) = I(\mathrm{round}(x), \mathrm{round}(y)).$$

Here round selects the nearest integer, using a fixed tie rule at half-integers, and the hat marks an interpolated estimate. This method preserves the original sample values, which is useful for discrete labels. For photographs, its piecewise-constant output often makes enlarged edges appear blocky.

Bilinear interpolation uses four surrounding samples. Let

$$x_0 = \lfloor x \rfloor, \quad y_0 = \lfloor y \rfloor, \qquad u = x - x_0, \quad v = y - y_0.$$

The floor operator, $\lfloor x \rfloor$, gives the greatest integer no larger than x; for example, $\lfloor 2.7 \rfloor = 2$. Thus (x_0, y_0) is the top-left sample of the cell containing the query, and $0 \le u, v < 1$. Denote the four intensities by

$$a = I(x_0, y_0), \quad b = I(x_0 + 1, y_0), \quad c = I(x_0, y_0 + 1), \quad d = I(x_0 + 1, y_0 + 1).$$

Figure 2 shows their positions. The quantities u and v are fractional offsets within one pixel cell, not image intensities.

Step 1: interpolate along each row. Linear interpolation between the left and right samples gives

$$q_1 = (1 - u)a + ub, \qquad q_2 = (1 - u)c + ud.$$

Step 2: interpolate between the rows. Weight the top and bottom interpolated values by their vertical proximity:

$$q = (1 - v)q_1 + vq_2.$$

Substitution produces the four-sample expression:

$$\boxed{\widehat{I}(x, y) = (1 - u)(1 - v)a + u(1 - v)b + (1 - u)vc + uvd.}$$

The weights are nonnegative and sum to one. A constant image remains constant, and the result lies between the smallest and largest of the four intensities.

5.4 Understanding the interpolation weights

The weight assigned to a sample increases as the query moves toward it. In distance notation, let $d_1 = u$ be the distance from the left edge, $d_2 = 1 - u$ from the right edge, $d_3 = v$ from the top, and $d_4 = 1 - v$ from the bottom. Then

$$q_1 = a\,d_2 + b\,d_1, \qquad q_2 = c\,d_2 + d\,d_1, \qquad q = q_1 d_4 + q_2 d_3.$$

These distances are measured in units of the grid spacing, so $d_1 + d_2 = d_3 + d_4 = 1$.

Example: interpolating a pixel value. Suppose the four samples are $a = 20$, $b = 60$, $c = 40$, and $d = 100$, and the query has $u = 1/4$ and $v = 1/2$. Then

$$\begin{aligned} q_1 &= \tfrac{3}{4}(20) + \tfrac{1}{4}(60) = 30, \\ q_2 &= \tfrac{3}{4}(40) + \tfrac{1}{4}(100) = 55, \\ q &= \tfrac{1}{2}(30) + \tfrac{1}{2}(55) = 42.5. \end{aligned}$$

For colour images, apply the same weights to every channel. The spatial geometry is shared across channels.

The interpolation order does not matter. Interpolate vertically first to obtain the left and right values,

$$r_1 = (1 - v)a + vc, \qquad r_2 = (1 - v)b + vd.$$

Then interpolate horizontally:

$$\begin{aligned} q &= (1 - u)r_1 + ur_2 \\ &= (1 - u)(1 - v)a + (1 - u)vc + u(1 - v)b + uvd. \end{aligned}$$

These are the same four weighted samples as before. Thus either order gives the same result: linear interpolation along each coordinate is what makes the method bilinear.

5.5 Bicubic interpolation and boundary handling

Bicubic interpolation replaces the linear interpolation along each coordinate with a cubic rule. A common implementation uses a 4×4 neighbourhood. It can give smoother intensity changes than bilinear interpolation, although the result depends on the chosen cubic kernel and may overshoot the sample range near sharp edges. It does not recover new scene detail.

All interpolation rules need a boundary policy when their neighbourhood extends beyond the image. Typical choices include a constant background or extending the nearest boundary value. Use a consistent rule, including when a query lies exactly on the last row or column.

Table 2: Core formulas for image transformations, camera geometry, and image sampling.

Concept	Core formula
Homogeneous point	$\widetilde{\mathbf{x}} = (x, y, 1)^\top \sim \lambda\widetilde{\mathbf{x}}$, $\lambda \neq 0$
Dehomogenization	$\pi(\mathbf{q}) = (q_1/q_3, q_2/q_3)^\top$, $q_3 \neq 0$
Translation	$\mathbf{x}' = \mathbf{x} + \mathbf{t}$
Euclidean transformation	$\mathbf{x}' = \mathbf{R}\mathbf{x} + \mathbf{t}$, $\mathbf{R}^\top\mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$
Similarity	$\mathbf{x}' = \alpha\mathbf{R}\mathbf{x} + \mathbf{t}$, $\alpha > 0$
Affine transformation	$\mathbf{x}' = \mathbf{A}\mathbf{x} + \mathbf{t}$, $\det \mathbf{A} \neq 0$
Homography	$\widetilde{\mathbf{x}}' \sim \mathbf{H}\widetilde{\mathbf{x}}$, $\det \mathbf{H} \neq 0$; eight degrees of freedom
Composition	Apply $\mathbf{H}_1$ then $\mathbf{H}_2$: $\mathbf{H}_{\text{total}} = \mathbf{H}_2\mathbf{H}_1$
Cross ratio	$C = \dfrac{(s_3 - s_1)(s_4 - s_2)}{(s_3 - s_2)(s_4 - s_1)}$
Point–line incidence	$\mathbf{l}^\top\widetilde{\mathbf{x}} = 0$
Line transformation	$\mathbf{l}' \sim \mathbf{H}^{-\top}\mathbf{l}$
Plane-to-image map	$\mathbf{H} = \mathbf{M}_{\text{int}}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$, for world plane $Z = 0$
Calibrated planar pose	$\mathbf{B} = \mathbf{M}_{\text{int}}^{-1}\widehat{\mathbf{H}}$; $\mathbf{r}_1 = \lambda\mathbf{b}_1$, $\mathbf{r}_2 = \lambda\mathbf{b}_2$, $\mathbf{t} = \lambda\mathbf{b}_3$, $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2$
Two views of a plane	$\mathbf{H}_{1\leftarrow 2} = \mathbf{H}_1\mathbf{H}_2^{-1}$
Pure rotation	$\mathbf{H}_{1\leftarrow 2} = \mathbf{M}_{\text{int}}\mathbf{R}\mathbf{M}_{\text{int}}^{-1}$, for identical intrinsics
Inverse warp	$I_{\text{out}}(\mathbf{x}') = \widehat{I}_{\text{in}}\big(\pi(\mathbf{T}^{-1}\widetilde{\mathbf{x}}')\big)$. Here $\mathbf{T}$ is an invertible 3×3 source-to-output transformation matrix; π divides the first two homogeneous components by the third to recover image coordinates.
Bilinear interpolation	$q = (1-u)(1-v)a + u(1-v)b + (1-u)vc + uvd$

6 Summary of Core Formulas

Table 2 collects the principal formulas developed in this note.

7 Common Pitfalls

1. **Forgetting homogeneous division.** The first two entries of $\mathbf{H}\widetilde{\mathbf{x}}$ are not generally the final image coordinates.
2. **Confusing points and lines.** Points use $\mathbf{H}$; lines use $\mathbf{H}^{-\top}$.
3. **Losing mapping direction or order.** Name the source and destination; the rightmost matrix acts first.
4. **Overextending the plane model.** One homography maps corresponding image points on a single scene plane between two views. It also relates views of a general scene under pure camera rotation, but camera translation generally prevents one homography from aligning points at different depths.
5. **Confusing sampling with geometry.** Inverse warping still needs interpolation and boundary handling.